Quantstamp DRAFT

SapienVault

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	VAULT
Timeline	2026-05-18 through 2026-05-20
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Source Code	• Sapien-io/sapien-contracts #e9ecac2
Auditors	• Andrei Stefan Auditing Engineer • Gereon Mendler Auditing Engineer

Documentation quality	High
Test quality	High
Total Findings	7 Unresolved: 7
High severity findings	1 Unresolved: 1
Medium severity findings	2 Unresolved: 2
Low severity findings	1 Unresolved: 1
Undetermined severity findings	0
Informational findings	3 Unresolved: 3

Summary of Findings

The `SapienVault` is a `ERC-4626` vault intended to track staking, controlled by a permissioned `Engine` role. The vault securely keeps all deposited assets, which remain locked and unused at all times in this contract. MEV protection is implemented by enforcing a multi-day delay between the most recent self-deposit or incoming transfer and the locking of a stake, withdraws or outgoing transfers. Users can lock their own stakes, but must rely on the Engine to unlock. The value of shares increases if stakers are slashed by burning some of their shares, hence share value strictly increases over time.

The `SapienVault` audit identified several unresolved issues centered on the vault's MEV/cooldown design and slashing accounting. The most severe issue is a high-risk (`SAP-1`) transfer griefing vector: because any incoming share transfer resets the recipient's global deposit-age timer, an attacker can repeatedly send dust shares to indefinitely block a user from transferring, staking, or redeeming their funds. A separate medium-severity (`SAP-2`) issue in the later review found that partial slashes are economically ineffective because slashed assets remain in the vault while only shares are burned, causing the loss to be redistributed back to all remaining shareholders, including the slashed user. This means a user can be slashed yet still recover nearly all of their original value through the increased share price. Additional weaknesses show that the cooldown can be bypassed through third-party deposits (`SAP-3`), allowing users to immediately use newly deposited shares and potentially exploit slash timing.

The core recommendation is to redesign the staking and cooldown model so it tracks individual stakes or tranches rather than applying a single global age restriction to an account's full balance. This would eliminate the transfer griefing issue, prevent users from bypassing protections via delegated deposits, and improve composability for `ERC-4626` integrations.

ID	DESCRIPTION	SEVERITY	STATUS
SAP-1	Shares Transfer Griefing Vector	High	Unresolved
SAP-2	Partial Slashes Are Self-Redistributed Back to the Slashed Holder	Medium	Unresolved
SAP-3	Mev: Protection Bypass via Delegate Deposits	Medium	Unresolved
SAP-4	Mev: Fresh Deposits Capture Slash Redistribution	Low	Unresolved

ID	DESCRIPTION	SEVERITY	STATUS
SAP-5	`minDepositAge` **Is Not Initialized**	• Informational	Unresolved
SAP-6	**Global Token Age Requirement May Punsish Active Users**	• Informational	Unresolved
SAP-7	**MEV Guard May Break Composibility**	• Informational	Unresolved

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

> **Disclaimer**
> Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/Sapien-io/sapien-contracts`

Included Paths: `src/`

Operational Considerations

1. **Third-party deposit documentation should be tightened**: `docs/SapienVault.md` claims every `deposit` / `mint` sets the receiver timestamp, but `_deposit` only does so when `caller == receiver`. The docs should match the code because this distinction matters for integrators and underpins one of the findings below.

2. **Pooled adapters need their own accounting**: A shared adapter address can have all vault-level exits delayed by a self-deposit, and address-scoped slashing can socialize one pooled user's penalty unless the adapter tracks per-user ownership and slash attribution.
3. **Raw ERC-4626 calls should not be slippage-blind**: Integrators should reject `previewDeposit == 0`, use min-share checks, and avoid depositing into a zero-supply/positive-assets state.
4. **Stake lifecycle is asymmetrically permissioned**: `lockStake` is permissionless once `minDepositAge` is met, but `unlockStake` and `slashStake` are gated to `ENGINE_ROLE`. Users cannot exit a locked position without engine cooperation, so engine liveness — not just engine honesty — is part of the trust model.
5. **Engine and admin keys are part of the security boundary**: `ENGINE_ROLE` (slash/unlock) and `DEFAULT_ADMIN_ROLE` (pause, upgrade, role management, `minDepositAge`, ETH rescue) configure or move user value directly. Both should be held behind multisig quorums and/or hardware wallets; key compromise materially expands the impact of every other finding in this report.

Key Actors And Their Capabilities

1. **Vault users and share holders**: Can deposit, mint, withdraw, redeem, transfer shares, approve spenders, and call `lockStake`. Withdrawals and transfers are constrained by `lockedAmount` and `minDepositAge`.
2. **ENGINE_ROLE**: Trusted role that can call `unlockStake` and `slashStake`. Findings rely on honest slashes interacting badly with vault accounting, not a malicious engine.
3. **DEFAULT_ADMIN_ROLE**: Trusted role that can set `minDepositAge`, pause/unpause, upgrade, rescue ETH, and manage roles. Privileged compromise is out of scope.
4. **Searchers or helper accounts**: Ordinary addresses that can provide capital, perform third-party ERC-4626 deposits, transfer shares, and order transactions around visible slashes.
5. **Integrators and pooled adapters**: Contracts holding vault shares for many users. They must account for address-scoped cooldowns, address-scoped locks, and ERC-4626 preview slippage.

Findings

SAP-1 Shares Transfer Griefing Vector

High Unresolved

File(s) affected: `SapienVault.sol`

Description: The `SapienVault` implements an MEV exploit protection mechanism that ensures that some configured time has elapsed after a deposit before other actions such as staking, transferring and redeeming/withdrawing can be executed. This lock applies globally to the entire token balance of a user. It is triggered by self-deposits, but also by incoming share transfers.

This vector can be exploitet by others, who can send dust amounts of shares frequently enough such that the recipient account never passes the MEV protection checks, locking all user funds in that account.

Recommendation: We suggest refactoring the MEV protection mechanism. We believe the fundamental intention is the protection of the staking mechanism, such that users may not stake right after deposit, and must remain staked for some duration of time. We suggest the following design, which tracks individual stakes: Instead of recording only the locked balance, each account holds a limited amount of individual stakes. Each stake enforces some minimum `stakeAmount`, as well as a `startDate` and optionally an earliest withdrawable `maturityDate` (currently enforced by the `Engine`). This list could then be squashed in-flight into an aggregated `stakedBalance` for stakes where the withdraw date has been exceeded. Additionally, this would allow users to withdraw unused assets right away if they are not used in stakes.

SAP-2 Partial Slashes Are Self-Redistributed Back to the Slashed Holder

Medium Unresolved

File(s) affected: `src/SapienVault.sol`

Description: `slashStake` reduces `lockedAmount` and calls `_burnShares(user, amount)`, which burns `previewWithdraw(amount)` shares . The burn shrinks supply but no SAPIEN leaves the vault. Since OZ's ERC-4626 `totalAssets()` is the raw token balance (`ERC4626Upgradeable.sol:138–140`), the slashed value stays in the pool and lifts the exchange rate for all remaining shares including those still held by the slashed user.

Exploit Scenario:

1. User deposits 1000 SAPIEN into an empty/attacker-dominated vault.
2. After `minDepositAge`, user locks 400 via `lockStake`.
3. Engine honestly calls `slashStake(user, 400e18)`.
4. Shares burn but 400 SAPIEN stays in `totalAssets()`.
5. User's remaining shares absorb the rate bump and redeem ~the full deposit.

Recommendation: Change the slashing function such that the intended damage is the input parameter, and the function properly calculates the dilution effect and attempts to burn up to this amount of shares from this user, if available.

SAP-3 Mev: Protection Bypass via Delegate Deposits

Medium | Unresolved

File(s) affected: `src/SapienVault.sol`

Description: The MEV protection mechanism is intended to apply for all balance changes, but is currently withheld for delegate deposits to avoid exploit scenarios. However, users can use this gap to circumvent this security barrier and make immediate use of deposited tokens, either locking stakes or griefing other users via transfers. Triggering `_deposit()` refreshes the deposit timestamp only when `caller == receiver` (src/SapienVault.sol:119-123), so a helper can mint shares to an aged receiver without resetting their cooldown. Minted shares enter via `_update(address(0), receiver, shares)`, which skips the wallet-to-wallet timestamp branch.

Exploit Scenario:

1. A malicious user can set up contract A to serve as passthrough and owner of the Vault Shares.
2. Deposits can be routed via another custom contract B to circumvent the self-deposit check.
3. A can then make immediate use of these tokens, since the `lastDepositTimestamp` of A is never updated.
4. Further, both contracts could be setup and executed atomically, facilitating MEV exploits

Recommendation: Ensure that the MEV protection holds equally for all balance modifying vectors. This would also reduce the severity of the transfer griefing vector, as it would make the attack significantly more complex by requiring advance planning. To avoid a new griefing vector, consider refactoring the staking along the recommendation outlined in `#SAP-1`.

SAP-4 Mev: Fresh Deposits Capture Slash Redistribution

Low | Unresolved

File(s) affected: `src/SapienVault.sol`

Description: If a slash is pending or predictable, a MEV operator can deposit assets to capture the resulting exchange-rate increase. The age restriction bypass outlined in `#SAP-3` even allows for immediate redeems, simplifying the exploit. But even without this bypass, the vault design guarantees that share value is strictly increasing over time. The only risk to an attacker is the opportunity cost of the assets locked in this vault before they can be withdrawn

Exploit Scenario:

1. `minDepositAge` is nonzero; attacker controls helper plus an aged/timestamp-free receiver.
2. A slash is visible, predictable, or orderable in the same block.
3. Helper calls `deposit(A, receiver)` before the slash; receiver gets shares without a fresh timestamp.
4. Engine honestly slashes another account, lifting the exchange rate.
5. Receiver immediately redeems and captures redistribution that should have gone to incumbents.

Recommendation: We suggest delaying the minting of shares on deposits. One possible design change would link the buying and redeeming of shares to locked stakes, such that only active stakes profit from redistribution. This would require some alteration of the usual `ERC-4626` behavior. Users would supply assets using a new entrypoint, and can later use this pending `Balance` to `Deposit` or `Mint` via the regular functions, now equivalent with the locking of a stake. Since this two step process excludes some assets from the share accounting, the internal `ERC-4626` needs to be overridden to not use the total asset balance of the vault to determine share value

SAP-5 `minDepositAge` Is Not Initialized

Informational | Unresolved

File(s) affected: `SapienVault.sol`

Description: The MEV protection mechanism relies on the configurable `minDepositAge` parameter. This can be set by the `admin`, but is not initialized during setup. Consequently, the `minDepositAge` is initially set to 0, and thus the MEV protection is disabled.

Recommendation: Set the `minDepositAge` to some value defined as `constant` or supplied as paramter in the `initialize()` function, and emit a corresponding `MinDepositAgeUpdated` event.

SAP-6 Global Token Age Requirement May Punsish Active Users

Informational | Unresolved

File(s) affected: `SapienVault.sol`

Description: The MEV protection mechanism is intended to apply for all balance changes and is currently applied to the entire balance of a user. Not only does it enforce some delay between deposit and staking, it also blocks redeems of recently unlocked stakes if there is deposit overlap. This design requires substantial planning and poses integration difficulties for very active users, who need to ensure a low frequency of deposits that aligns with any withdraw schedule. Further, users may not be able to control the unstake time, as it is a permissioned action only executable by the `Engine`.

Recommendation: Refactor the MEV protection to allow withdraws independently from deposits. The suggested staking refactoring would fully mitigate this issue.

SAP-7 MEV Guard May Break Composibility

Informational | Unresolved

File(s) affected: `SapienVault.sol`

Description: ERC-4626 vaults are often integrated with other DeFi protocols, such that funds are automatically deposited and used by other contracts. The current MEV protection mechanism may break this composibility by blocking the immediate use of deposited or transferred tokens. If a user routes funds through an intermediary contract like an auto-compounder, router, or multi-step zapper for deposit and tries to do anything else with the shares in the same transaction, the transaction will revert.

Recommendation: Refactoring the staking system as suggested fully mitigates this issue. Alternatively, document this behavior.

Auditor Suggestions

S1 Expose Explicit Cooldown State for Users and Integrators

Unresolved

File(s) affected: `src/SapienVault.sol`

Description: `lastDepositTimestamp` drives MEV/cooldown enforcement but is only used internally via `_hasMetDepositAge` / `_requireDepositAgeMet` (`src/Types.sol:10`, `src/SapienVault.sol:90–106`). The public surface exposes `minDepositAge`, `availableBalance`, `maxWithdraw`, `maxRedeem`, and `getStakeAccount`, but not the per-user timestamp or remaining cooldown. When `maxWithdraw(owner) == 0`, an integrator cannot distinguish paused, locked, in cooldown, no-shares, or other caps.

Recommendation: Expose cooldown read helpers. Minimal:

```
function lastDepositTimestamp(address user) external view returns (uint256);
```

Fuller status helper:

```
function depositAgeStatus(address user)
    external
    view
    returns (bool hasMetAge, uint256 lastTimestamp, uint256 minAge, uint256 remaining);
```

Also emit both old and new values in `MinDepositAgeUpdated` so off-chain monitoring can reconstruct config changes without an extra read.

S2 Improved Role Management

Unresolved

File(s) affected: `SapienVault.sol`

Description: Consider switching from the standard `AccessControlUpgradeable` library to `AccessControlDefaultAdminRulesUpgradeable` to improve the security and robustness of role management. This includes two-step admin role transfers with time locks and strict single-admin enforcement. Additionally, we recommend overriding the ability to renounce ownership altogether.

S3 Code Improvements

Unresolved

File(s) affected: `SapienVault.sol`, `Types.sol`

Description: We suggest the following general code improvements:

1. In `Types.sol`, the `StakeAccount` struct is not strictly needed. Instead, use `uint256` directly or store `lastDepositTimestamp` here.
2. In `SapienVault.sol`, avoid code duplication between the `_hasMetDepositAge()` and `_requireDepositAgeMet()` functions.
3. In `SapienVault.sol`, avoid code duplication between the `maxRedeem()` and `maxWithdraw()` functions.
4. To avoid misleading names, rename the `getUserStakeBalance()` function, which returns the total asset value of the user, not the staked amount.
5. In `SapienVault.sol`, the `setMinDepositAge()` function emits an event even if the new `age` is equal to the stored `minDepositAge`.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.

- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/Sapien-io/sapien-contracts`

- `3b3...9d8 ./src/SapienVault.sol`
- `2e6...698 ./src/Types.sol`
- `a40...b22 ./src/interfaces/ISapienVault.sol`

Test Suite Results

`forge test`

```
Ran 2 tests for test/MockUSDC.t.sol:MockUSDCTest
[PASS] test_decimals_isSix() (gas: 421408)
[PASS] test_mint() (gas: 468443)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 6.88ms (3.20ms CPU time)

Ran 7 tests for test/SapienVaultInvariant.t.sol:SapienVaultInvariantTest
[PASS] invariant_ExchangeRateNeverDecreases() (runs: 1024, calls: 51200, reverts: 6418)
[PASS] invariant_GhostLockedMatchesActual() (runs: 1024, calls: 51200, reverts: 5919)
[PASS] invariant_LockedNeverExceedsTotal() (runs: 1024, calls: 51200, reverts: 6334)
[PASS] invariant_TimeLockSafety() (runs: 1024, calls: 51200, reverts: 6360)
[PASS] invariant_TotalAssetsGteTotalLocked() (runs: 1024, calls: 51200, reverts: 6380)
[PASS] invariant_UserSharesCoverLocked() (runs: 1024, calls: 51200, reverts: 6238)
[PASS] invariant_VaultTokenBalance() (runs: 1024, calls: 51200, reverts: 6457)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 6.61s (26.48s CPU time)

Ran 102 tests for test/SapienVault.t.sol:SapienVaultTest
[PASS] testFuzz_deposit(uint256) (runs: 256, μ: 147970, ~: 147759)
[PASS] testFuzz_lockStake(uint256,uint256) (runs: 256, μ: 181222, ~: 181093)
[PASS] testFuzz_maxRedeem_preservesLockInvariant(uint256,uint256,uint256) (runs: 256, μ: 219807, ~:
220516)
[PASS] testFuzz_maxWithdraw_preservesLockInvariant(uint256,uint256,uint256) (runs: 256, μ: 226454, ~:
227040)
[PASS] testFuzz_mint(uint256) (runs: 256, μ: 147066, ~: 146928)
[PASS] testFuzz_redeem(uint256,uint256) (runs: 256, μ: 171961, ~: 172553)
[PASS] testFuzz_rescueETH_anyBalance(uint96) (runs: 256, μ: 57685, ~: 57685)
[PASS] testFuzz_setMinDepositAge(uint256) (runs: 256, μ: 48110, ~: 48685)
[PASS] testFuzz_slashStake(uint256,uint256,uint256) (runs: 256, μ: 193427, ~: 195431)
[PASS] testFuzz_slashStake_alwaysBurnsShares(uint256,uint256,uint256) (runs: 256, μ: 185672, ~: 185606)
[PASS] testFuzz_slashStake_preservesLockInvariant(uint256,uint256,uint256,uint256) (runs: 256, μ: 209572,
~: 210400)
[PASS] testFuzz_transfer(uint256,uint256,uint256) (runs: 256, μ: 236494, ~: 238705)
```

```
[PASS] testFuzz_transferGuard_roundUp_invariant(uint256,uint256,uint256) (runs: 256, μ: 243164, ~:
249675)
[PASS] testFuzz_unlockStake(uint256,uint256,uint256) (runs: 256, μ: 184037, ~: 185422)
[PASS] testFuzz_withdraw(uint256,uint256) (runs: 256, μ: 168710, ~: 169618)
[PASS] test_adminCanPause() (gas: 36136)
[PASS] test_adminRenounce_blocksAdminOps() (gas: 27895)
[PASS] test_authorizeUpgrade() (gas: 2295981)
[PASS] test_bypass_warmSybilTransfer() (gas: 227216)
[PASS] test_cannotReinitialize() (gas: 22603)
[PASS] test_cannotWithdrawLockedFunds() (gas: 169071)
[PASS] test_decimalsOffset() (gas: 18084)
[PASS] test_deposit() (gas: 135619)
[PASS] test_depositOnBehalf_doesNotResetTimestamp() (gas: 221676)
[PASS] test_deposit_oneWei() (gas: 142004)
[PASS] test_donationAttack_firstDepositor() (gas: 158330)
[PASS] test_donationDoesNotBreakExistingStakers() (gas: 152380)
[PASS] test_griefing_resetTimestampViaDustTransfer() (gas: 285208)
[PASS] test_initialize_revertsZeroAddress() (gas: 2426069)
[PASS] test_lockStake() (gas: 167728)
[PASS] test_lockStake_calledByOwnerNotEngine() (gas: 161141)
[PASS] test_lockStake_emitsStakeLocked() (gas: 161446)
[PASS] test_lockStake_entireBalance() (gas: 179905)
[PASS] test_lockStake_respectsMinDepositAge() (gas: 193197)
[PASS] test_lockStake_revertsForFreshTransferWhenMinDepositAgeSet() (gas: 255557)
[PASS] test_lockStake_revertsInsufficientBalance() (gas: 138510)
[PASS] test_lockStake_revertsWhenPaused() (gas: 176585)
[PASS] test_lockStake_revertsZeroAmount() (gas: 131360)
[PASS] test_lockUnlockLifecycle() (gas: 157964)
[PASS] test_maxMintAndDeposit_pausedAndUnpaused() (gas: 54160)
[PASS] test_maxRedeem_conservativeAfterDonation() (gas: 201824)
[PASS] test_maxRedeem_paused() (gas: 158317)
[PASS] test_maxRedeem_timeLockNotMet() (gas: 167808)
[PASS] test_maxWithdraw_conservativeAfterDonation() (gas: 210263)
[PASS] test_maxWithdraw_doesNotRevert_afterDonation() (gas: 211091)
[PASS] test_maxWithdraw_fullWithdraw_noLock() (gas: 176731)
[PASS] test_maxWithdraw_partialLock_postDonation() (gas: 212027)
[PASS] test_mev_frontrunSlashStake() (gas: 266346)
[PASS] test_minDepositAge_view() (gas: 45698)
[PASS] test_mint() (gas: 138605)
[PASS] test_mint_revertsWhenPaused() (gas: 47470)
[PASS] test_multipleEngines() (gas: 212561)
[PASS] test_multipleLockUnlockCycles() (gas: 239284)
[PASS] test_nonAdminCannotPause() (gas: 23290)
[PASS] test_onlyEngineCanSlashStake() (gas: 166765)
[PASS] test_onlyEngineCanUnlockStake() (gas: 168371)
[PASS] test_redeem() (gas: 119930)
[PASS] test_redeem_respectsLockedShares() (gas: 189782)
[PASS] test_redeem_respectsTimeLock() (gas: 143315)
[PASS] test_redeem_revertsWhenPaused() (gas: 162422)
[PASS] test_rescueETH_adminCanRecover() (gas: 60453)
[PASS] test_rescueETH_revertsForNonAdmin() (gas: 20071)
[PASS] test_rescueETH_revertsOnZeroAddress() (gas: 22793)
[PASS] test_rescueETH_revertsWhenNoBalance() (gas: 27466)
[PASS] test_rescueETH_revertsWhenRecipientRejects() (gas: 86713)
[PASS] test_revokeEngineRole_blocksOperations() (gas: 182577)
[PASS] test_selfTransfer_resetsOwnTimestamp() (gas: 176581)
[PASS] test_setMinDepositAge_emitsMinDepositAgeUpdated() (gas: 43838)
[PASS] test_setMinDepositAge_tooHigh() (gas: 22825)
[PASS] test_slashAfterPartialWithdrawal() (gas: 182530)
[PASS] test_slashDoesNotReduceOtherUserBalance() (gas: 238176)
[PASS] test_slashStake() (gas: 178994)
[PASS] test_slashStake_emitsStakeSlashed() (gas: 175013)
[PASS] test_slashStake_revertsInsufficientLock() (gas: 166785)
[PASS] test_slashStake_revertsWhenPaused() (gas: 191651)
[PASS] test_slashStake_revertsZeroAmount() (gas: 167645)
[PASS] test_slashStake_roundsUp_afterDonation() (gas: 202042)
[PASS] test_slashStake_roundsUp_burnsAtLeastOneShare() (gas: 181802)
[PASS] test_slashStake_smallSlash_highExchangeRate() (gas: 192598)
[PASS] test_slashStake_zeroShareSlash_reverts() (gas: 164930)
[PASS] test_thirdPartyDeposit_doesNotSetTimestamp() (gas: 179922)
[PASS] test_transferFrom_resetsReceiverTimestamp() (gas: 264708)
[PASS] test_transferFrom_respectsLockedShares() (gas: 200493)
```

```
[PASS] test_transferFrom_respectsPause() (gas: 188996)
[PASS] test_transferFrom_respectsTimeLock() (gas: 249340)
[PASS] test_transferGuard_allowsMaxUnlocked_postDonation() (gas: 245622)
[PASS] test_transferGuard_noLock_fullTransfer_postDonation() (gas: 188564)
[PASS] test_transferGuard_preventsUndercollateralisation_highExchangeRate() (gas: 190162)
[PASS] test_transferGuard_roundUpReservation_smallLock() (gas: 251937)
[PASS] test_transferRevertsWhenPaused() (gas: 163637)
[PASS] test_transfer_resetsTimestampOfExistingStaker() (gas: 237019)
[PASS] test_transfer_revertsTransferExceedsUnlockedShares() (gas: 174112)
[PASS] test_transfer_setsTimestampForFirstTimeRecipient() (gas: 254655)
[PASS] test_unlockStake() (gas: 170526)
[PASS] test_unlockStake_emitsStakeUnlocked() (gas: 171579)
[PASS] test_unlockStake_onlyEngine() (gas: 167315)
[PASS] test_unlockStake_revertsInsufficientLock() (gas: 168071)
[PASS] test_unlockStake_revertsWhenPaused() (gas: 194991)
[PASS] test_unlockStake_revertsZeroAmount() (gas: 167645)
[PASS] test_verifyStorageLocation() (gas: 15686)
[PASS] test_withdraw() (gas: 119696)
[PASS] test_zeroValueTransfer_doesNotResetTimestamp() (gas: 265173)
Suite result: ok. 102 passed; 0 failed; 0 skipped; finished in 6.61s (415.28ms CPU time)

Ran 3 test suites in 6.61s (13.22s CPU time): 111 tests passed, 0 failed, 0 skipped (111 total tests)
```

Changelog

- 2026-05-20 - Initial report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.